

Machine Learning: A Comprehensive Textbook

Solutions to Chapter 5 Exercises

Classification: From Logistic Regression to Support Vector Machines

Solutions Manual

Exercise 5.1 — Sigmoid Properties

Prove: (a) $\sigma'(z) = \sigma(z)(1 - \sigma(z))$; (b) $\sigma(-z) = 1 - \sigma(z)$; (c) $\log \sigma(z) = -\log(1 + e^{-z})$.

Solution. Recall $\sigma(z) = \frac{1}{1 + e^{-z}}$.

(a) Differentiate using the quotient/chain rule: $\sigma(z) = (1 + e^{-z})^{-1}$, so

$$\sigma'(z) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}.$$

Write $\frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z) \cdot \frac{e^{-z}}{1 + e^{-z}}$. Now note $\frac{e^{-z}}{1 + e^{-z}} = \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} = 1 - \frac{1}{1 + e^{-z}} = 1 - \sigma(z)$. Hence $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. ■

(b)

$$\sigma(-z) = \frac{1}{1 + e^z} = \frac{e^{-z}}{e^{-z}(1 + e^z)} = \frac{e^{-z}}{e^{-z} + 1} = \frac{e^{-z}}{1 + e^{-z}} = 1 - \sigma(z)$$

using the identity $\frac{e^{-z}}{1 + e^{-z}} = 1 - \sigma(z)$ derived in part (a). ■

(c)

$$\log \sigma(z) = \log \left(\frac{1}{1 + e^{-z}} \right) = -\log(1 + e^{-z}). \quad \blacksquare$$

(This identity is numerically important: computing $\log \sigma(z)$ directly can overflow/underflow for large $|z|$, while $-\log(1 + e^{-z})$, combined with the `softplus/log-sum-exp` trick, is numerically stable — this is exactly why frameworks implement fused `log_sigmoid` or `binary_cross_entropy_with_logits` functions.)

Exercise 5.2 — Log-Loss Gradient

Starting from $J(\theta) = -\frac{1}{n} \sum_i [y^{(i)} \log \hat{p}_i + (1 - y^{(i)}) \log(1 - \hat{p}_i)]$, derive $\nabla_{\theta} J = \frac{1}{n} X^{\top} (\hat{p} - y)$ step by step.

Solution. Let $\hat{p}_i = \sigma(\theta^{\top} x^{(i)})$, and write $z_i = \theta^{\top} x^{(i)}$. By the chain rule, for a single example:

$$\frac{\partial}{\partial \theta} \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] = \left(\frac{y_i}{\hat{p}_i} - \frac{1 - y_i}{1 - \hat{p}_i} \right) \cdot \frac{\partial \hat{p}_i}{\partial \theta}.$$

Using $\partial \hat{p}_i / \partial \theta = \sigma'(z_i) x^{(i)} = \hat{p}_i(1 - \hat{p}_i) x^{(i)}$ (Exercise 5.1a):

$$\left(\frac{y_i}{\hat{p}_i} - \frac{1 - y_i}{1 - \hat{p}_i} \right) \hat{p}_i(1 - \hat{p}_i) x^{(i)} = \left[y_i(1 - \hat{p}_i) - (1 - y_i)\hat{p}_i \right] x^{(i)}.$$

Expand the bracket: $y_i - y_i\hat{p}_i - \hat{p}_i + y_i\hat{p}_i = y_i - \hat{p}_i$. So

$$\frac{\partial}{\partial \theta} \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] = (y_i - \hat{p}_i) x^{(i)}.$$

Summing over i and including the $-\frac{1}{n}$ from J :

$$\nabla_{\theta} J = -\frac{1}{n} \sum_{i=1}^n (y_i - \hat{p}_i) x^{(i)} = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - y_i) x^{(i)}.$$

In matrix form, stacking $x^{(i)}$ as rows of X and $(\hat{p}_i - y_i)$ as entries of $(\hat{p} - y)$:

$$\nabla_{\theta} J = \frac{1}{n} X^{\top} (\hat{p} - y). \quad \blacksquare$$

(Remarkably, this has the exact same form as the OLS gradient $\frac{1}{n} X^{\top} (X\theta - y)$ from Exercise 4.1, with $X\theta$ replaced by $\sigma(X\theta)$ — a general property of generalized linear models with canonical link functions.)

Exercise 5.3 — Convexity of Logistic Loss

Prove the Hessian is $H = \frac{1}{n} X^{\top} D X$, $D = \text{diag}(\hat{p}_i(1 - \hat{p}_i))$, and show $H \succeq 0$.

Solution. From Exercise 5.2, $\nabla_{\theta} J = \frac{1}{n} \sum_i (\hat{p}_i - y_i) x^{(i)}$. Differentiate again with respect to θ :

$$\frac{\partial}{\partial \theta} [(\hat{p}_i - y_i) x^{(i)}] = x^{(i)} \cdot \frac{\partial \hat{p}_i}{\partial \theta}^{\top} = x^{(i)} (\hat{p}_i(1 - \hat{p}_i) x^{(i)})^{\top} = \hat{p}_i(1 - \hat{p}_i) x^{(i)} x^{(i)\top}.$$

Summing over i (with the $\frac{1}{n}$):

$$H = \nabla_{\theta}^2 J = \frac{1}{n} \sum_{i=1}^n \hat{p}_i(1 - \hat{p}_i) x^{(i)} x^{(i)\top}.$$

Writing $D = \text{diag}(\hat{p}_1(1 - \hat{p}_1), \dots, \hat{p}_n(1 - \hat{p}_n))$ and stacking $x^{(i)}$ as rows of X , this sum is exactly the matrix product

$$H = \frac{1}{n} X^{\top} D X.$$

PSD. For any vector $v \in \mathbb{R}^d$,

$$v^{\top} H v = \frac{1}{n} v^{\top} X^{\top} D X v = \frac{1}{n} (Xv)^{\top} D (Xv) = \frac{1}{n} \sum_{i=1}^n \hat{p}_i(1 - \hat{p}_i) (Xv)_i^2.$$

Each term in the sum is a product of $\hat{p}_i(1 - \hat{p}_i) \geq 0$ (since $\hat{p}_i \in (0, 1)$, so both factors are non-negative — in fact strictly positive for $\hat{p}_i \in (0, 1)$) and a square $(Xv)_i^2 \geq 0$. Hence every term is ≥ 0 , so $v^{\top} H v \geq 0$ for all v , i.e. $H \succeq 0$. $\blacksquare$

Since the Hessian is PSD everywhere (for any θ , since D always has non-negative diagonal entries as $\hat{p}_i(1 - \hat{p}_i) \geq 0$ regardless of θ), the logistic loss $J(\theta)$ is **convex** — this guarantees that gradient descent (with appropriate step size) converges to the global minimum, and that any local minimum found is global.

Exercise 5.4 — SVM Dual Derivation

Derive the hard-margin SVM dual (5.7): (a) KKT stationarity for w, b ; (b) substitute back; (c) simplify.

Solution. The primal is $\min_{w,b} \frac{1}{2}\|w\|^2$ s.t. $y^{(i)}(w^\top x^{(i)} + b) \geq 1$ for all i . The Lagrangian (with multipliers $\alpha_i \geq 0$) is

$$L(w, b, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^n \alpha_i [y^{(i)}(w^\top x^{(i)} + b) - 1].$$

(a) **Stationarity.**

$$\frac{\partial L}{\partial w} = w - \sum_i \alpha_i y^{(i)} x^{(i)} = 0 \implies w = \sum_i \alpha_i y^{(i)} x^{(i)},$$

$$\frac{\partial L}{\partial b} = - \sum_i \alpha_i y^{(i)} = 0 \implies \sum_i \alpha_i y^{(i)} = 0.$$

(b) **Substitute back.** Plug $w = \sum_i \alpha_i y^{(i)} x^{(i)}$ into L :

$$\frac{1}{2}\|w\|^2 = \frac{1}{2} \left(\sum_i \alpha_i y^{(i)} x^{(i)} \right)^\top \left(\sum_j \alpha_j y^{(j)} x^{(j)} \right) = \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)\top} x^{(j)}.$$

For the second term,

$$\sum_i \alpha_i y^{(i)} (w^\top x^{(i)} + b) = \sum_i \alpha_i y^{(i)} w^\top x^{(i)} + b \sum_i \alpha_i y^{(i)} = w^\top \left(\sum_i \alpha_i y^{(i)} x^{(i)} \right) + b \cdot 0$$

(using stationarity in b) $= w^\top w = \|w\|^2 = \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)\top} x^{(j)}.$

So

$$L = \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)\top} x^{(j)} - \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)\top} x^{(j)} + \sum_i \alpha_i.$$

(c) **Simplify.** The first two terms combine to $-\frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)\top} x^{(j)}$, giving the dual objective

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)\top} x^{(j)} \quad \text{s.t. } \alpha_i \geq 0, \sum_i \alpha_i y^{(i)} = 0,$$

which is precisely equation (5.7) — the SVM dual. Note the objective depends on the data only through inner products $x^{(i)\top} x^{(j)}$, which is exactly what enables the kernel trick (replacing the inner product with a kernel $k(x^{(i)}, x^{(j)})$, cf. Exercise 5.6).

Exercise 5.5 — Support Vectors via Complementary Slackness

Using $\alpha_i [y^{(i)}(w^{\star\top} x^{(i)} + b^*) - 1] = 0$, prove (a) $\alpha_i > 0 \implies$ example on the margin; (b) strictly-inside example $\implies \alpha_i = 0$.

Solution. Complementary slackness (a KKT condition) states $\alpha_i [y^{(i)}(w^{\star\top} x^{(i)} + b^*) - 1] = 0$ for every i , meaning for each i , *at least one* of the two factors must be zero.

(a) Suppose $\alpha_i > 0$. Since the product is zero and $\alpha_i \neq 0$, the other factor must vanish:

$$y^{(i)}(w^{*\top}x^{(i)} + b^*) - 1 = 0 \implies y^{(i)}(w^{*\top}x^{(i)} + b^*) = 1.$$

This is precisely the condition that example i lies *exactly on the margin boundary* (functional margin exactly 1), i.e. it is a support vector. ■

(b) Suppose example i lies *strictly inside* the margin, meaning $y^{(i)}(w^{*\top}x^{(i)} + b^*) > 1$ (strictly greater than 1 — feasible but not tight). Then the second factor $y^{(i)}(w^{*\top}x^{(i)} + b^*) - 1 > 0$ is strictly positive (nonzero). For the product $\alpha_i \cdot [\dots] = 0$ to hold with the second factor nonzero, we must have $\alpha_i = 0$. ■

Interpretation. Together, (a) and (b) establish the contrapositive-equivalence: $\alpha_i > 0 \iff$ example i is exactly on the margin (a support vector), and any example with positive margin (strictly correctly and confidently classified, off the margin boundary) necessarily has $\alpha_i = 0$ — it contributes nothing to $w^* = \sum_i \alpha_i y^{(i)} x^{(i)}$ (Exercise 5.4a). This is precisely why the SVM decision boundary depends only on the (typically small) set of support vectors, not on the full training set.

Exercise 5.6 — RBF Kernel Validity

Show $k(x, x') = \exp(-\|x - x'\|^2/2\sigma^2)$ satisfies Mercer's condition.

Solution. Expand the squared distance: $\|x - x'\|^2 = \|x\|^2 - 2x^\top x' + \|x'\|^2$. So

$$k(x, x') = \exp\left(-\frac{\|x\|^2 - 2x^\top x' + \|x'\|^2}{2\sigma^2}\right) = \exp\left(-\frac{\|x\|^2}{2\sigma^2}\right) \exp\left(\frac{x^\top x'}{\sigma^2}\right) \exp\left(-\frac{\|x'\|^2}{2\sigma^2}\right).$$

Define $g(x) := \exp(-\|x\|^2/2\sigma^2)$, a scalar function. Then $k(x, x') = g(x)\tilde{k}(x, x')g(x')$ where $\tilde{k}(x, x') = \exp(x^\top x'/\sigma^2)$.

Step 1: $\tilde{k}(x, x') = \exp(x^\top x'/\sigma^2)$ is a valid (Mercer) kernel. Using the Taylor/power series expansion of $\exp$:

$$\tilde{k}(x, x') = \exp\left(\frac{x^\top x'}{\sigma^2}\right) = \sum_{m=0}^{\infty} \frac{1}{m!} \left(\frac{x^\top x'}{\sigma^2}\right)^m = \sum_{m=0}^{\infty} \frac{1}{m!\sigma^{2m}} (x^\top x')^m.$$

Each term $(x^\top x')^m$ is the m -th power of the (trivially valid) linear kernel $x^\top x'$. The product of valid kernels is a valid kernel (Mercer kernels are closed under pointwise multiplication — this follows because if k_1, k_2 correspond to feature maps ϕ_1, ϕ_2 , then $k_1 k_2$ corresponds to the tensor-product feature map $\phi_1 \otimes \phi_2$, whose inner product recovers $k_1(x, x')k_2(x, x')$); hence $(x^\top x')^m$ is a valid kernel for each integer $m \geq 0$ (as an m -fold product of the linear kernel with itself). A non-negative-weighted sum (here, an infinite series with positive coefficients $1/(m!\sigma^{2m}) > 0$) of valid kernels is itself a valid kernel (this follows directly from the definition: the Gram matrix of a sum is the sum of Gram matrices, each PSD, and a non-negative combination of PSD matrices is PSD; the argument extends to the convergent infinite sum by continuity/limits of PSD matrices). Hence $\tilde{k}(x, x')$ is a valid Mercer kernel.

Step 2: multiplying by $g(x)g(x')$ preserves validity. For any valid kernel $\tilde{k}$ and any scalar function g , the product $k(x, x') = g(x)\tilde{k}(x, x')g(x')$ is also a valid kernel: if $\tilde{k}(x, x') = \phi(x)^\top \phi(x')$ for some feature map ϕ , then $k(x, x') = (g(x)\phi(x))^\top (g(x')\phi(x'))$, exhibiting k as an inner product under the feature map $\psi(x) := g(x)\phi(x)$ — explicitly, its Gram matrix is $K = \text{diag}(g)\tilde{K}\text{diag}(g)$, a congruence transform of the PSD matrix $\tilde{K}$, which preserves PSD-ness ($v^\top K v = (\text{diag}(g)v)^\top \tilde{K}(\text{diag}(g)v) \geq 0$ since $\tilde{K} \succeq 0$).

Combining both steps, $k(x, x') = g(x)\tilde{k}(x, x')g(x')$ is a valid Mercer kernel — i.e. its Gram matrix is PSD for any finite set of points $\{x^{(i)}\}$, so the RBF kernel satisfies Mercer's condition. ■

Exercise 5.7 — Naive Bayes MLE

Derive the Multinomial NB MLE $\hat{\theta}_{jk} = n_{jk}/n_k$. Show Laplace smoothing corresponds to a Dirichlet prior.

Solution. MLE derivation. For class k , let $\theta_{jk} = P(\text{word } j \mid \text{class } k)$, with $\sum_j \theta_{jk} = 1$. Let n_{jk} be the count of word j appearing across all documents of class k , and $n_k = \sum_j n_{jk}$ the total word count in class k . The (multinomial) log-likelihood of the class- k documents, as a function of $\theta_{\cdot k} = (\theta_{1k}, \dots, \theta_{V_k})$, is (up to an additive multinomial-coefficient constant not depending on θ):

$$\ell(\theta_{\cdot k}) = \sum_j n_{jk} \log \theta_{jk}.$$

We maximize this subject to the simplex constraint $\sum_j \theta_{jk} = 1$, using a Lagrange multiplier μ :

$$\mathcal{L} = \sum_j n_{jk} \log \theta_{jk} - \mu \left(\sum_j \theta_{jk} - 1 \right).$$

Setting $\partial \mathcal{L} / \partial \theta_{jk} = 0$: $\frac{n_{jk}}{\theta_{jk}} - \mu = 0 \implies \theta_{jk} = \frac{n_{jk}}{\mu}$. Summing over j and using the constraint: $\sum_j \theta_{jk} = \frac{1}{\mu} \sum_j n_{jk} = \frac{n_k}{\mu} = 1 \implies \mu = n_k$. Hence

$$\hat{\theta}_{jk} = \frac{n_{jk}}{n_k}. \quad \blacksquare$$

Laplace smoothing as a Dirichlet prior. Place a symmetric Dirichlet prior $\theta_{\cdot k} \sim \text{Dir}(\beta, \dots, \beta)$ (concentration β for each of the V vocabulary words) on $\theta_{\cdot k}$. Its density is $p(\theta_{\cdot k}) \propto \prod_j \theta_{jk}^{\beta-1}$. The multinomial likelihood is $p(\text{data} \mid \theta_{\cdot k}) \propto \prod_j \theta_{jk}^{n_{jk}}$ (Dirichlet is the conjugate prior to the multinomial). The posterior is therefore

$$p(\theta_{\cdot k} \mid \text{data}) \propto \prod_j \theta_{jk}^{n_{jk} + \beta - 1},$$

which is itself a $\text{Dir}(n_{1k} + \beta, \dots, n_{V_k} + \beta)$ distribution. The MAP estimate (mode of a Dirichlet($\alpha_1, \dots, \alpha_V$) distribution) is $\hat{\theta}_{jk} = \frac{\alpha_j - 1}{\sum_{j'} \alpha_{j'} - V}$ when all $\alpha_j > 1$; here $\alpha_j = n_{jk} + \beta$, giving

$$\hat{\theta}_{jk}^{\text{MAP}} = \frac{n_{jk} + \beta - 1}{n_k + V\beta - V} = \frac{n_{jk} + (\beta - 1)}{n_k + V(\beta - 1)}.$$

Setting $\beta = 2$ (i.e. a Dirichlet prior with $\beta - 1 = 1$ “pseudo-count” per word) recovers exactly Laplace (add-one) smoothing:

$$\hat{\theta}_{jk} = \frac{n_{jk} + 1}{n_k + V}.$$

More generally, add- α smoothing $\hat{\theta}_{jk} = (n_{jk} + \alpha)/(n_k + V\alpha)$ corresponds to a Dirichlet prior with concentration $\beta = \alpha + 1$ (using the posterior-mean rather than posterior-mode estimator, which for Dirichlet is $\mathbb{E}[\theta_{jk} \mid \text{data}] = \frac{n_{jk} + \beta}{n_k + V\beta}$, directly giving add- β smoothing without the -1 offset

— this is in fact the more standard way Laplace/add- α smoothing is presented, as the posterior mean under a $\text{Dir}(\alpha + 1, \dots, \alpha + 1)$ or, more simply, treating the smoothing constant directly as the Dirichlet concentration in the posterior-mean formulation). Either way, Laplace smoothing is precisely Bayesian MAP/posterior-mean estimation under a Dirichlet prior that regularizes the multinomial parameters away from zero, preventing the classic “unseen word $\Rightarrow$ zero probability $\Rightarrow$ entire document probability collapses to zero” failure of the raw MLE.

Exercise 5.8 — Decision Tree Splits

Parent: 10 pos, 10 neg. S_L : 8 pos, 2 neg. S_R : 2 pos, 8 neg. Compute (a) parent Gini/entropy, (b) child Gini/entropy, (c) Gini reduction/info gain, (d) which criterion favours this split more.

Solution. (a) **Parent node.** $p_+ = p_- = 10/20 = 0.5$.

$$\text{Gini}_{\text{parent}} = 1 - \sum_c p_c^2 = 1 - (0.5^2 + 0.5^2) = 1 - 0.5 = 0.5.$$

$$H_{\text{parent}} = - \sum_c p_c \log_2 p_c = -(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = -\log_2 0.5 = 1 \text{ bit.}$$

(b) **Children.** S_L : $p_+ = 8/10 = 0.8, p_- = 0.2$. S_R : $p_+ = 0.2, p_- = 0.8$ (symmetric to S_L).

$$\text{Gini}(S_L) = 1 - (0.8^2 + 0.2^2) = 1 - (0.64 + 0.04) = 1 - 0.68 = 0.32 = \text{Gini}(S_R) \text{ (by symmetry).}$$

$$H(S_L) = -(0.8 \log_2 0.8 + 0.2 \log_2 0.2) = -(0.8(-0.3219) + 0.2(-2.3219)) = 0.2575 + 0.4644 = 0.7219 \text{ bits} = H(S_R).$$

(c) **Weighted reduction.** Both children have equal size (10 each), so the weighted average child impurity is just the (equal) child value.

$$\Delta \text{Gini} = \text{Gini}_{\text{parent}} - \frac{1}{2} [\text{Gini}(S_L) + \text{Gini}(S_R)] = 0.5 - 0.32 = \mathbf{0.18}.$$

$$\text{IG} = H_{\text{parent}} - \frac{1}{2} [H(S_L) + H(S_R)] = 1 - 0.7219 = \mathbf{0.2781} \text{ bits.}$$

(d) **Comparing criteria.** To compare on a common scale, normalize each by its parent value: Gini reduction is $0.18/0.5 = 36.0\%$ of parent Gini eliminated; information gain is $0.2781/1 = 27.8\%$ of parent entropy eliminated. **Gini impurity favours this particular split more** (proportionally larger reduction) than entropy/information gain does. This reflects a general, well-known tendency: Gini impurity is (locally, near $p = 0.5$) more sensitive to changes that push a node toward purity than entropy is, since entropy has a “flatter” (more logarithmically-compressed) shape near the extremes and a comparatively sharper penalty in the middle — in practice the two criteria very often agree on which split is best among a set of candidates, but Gini is computationally cheaper (no logarithms) and is CART’s default, while ID3/C4.5 use entropy/information gain.

Exercise 5.9 — Curse of Dimensionality

Features uniform on $[0, 1]^d$, need 10 NN among $n = 1000$. What edge length must a hypercube cover? Compute for $d = 1, 2, 5, 10, 100$.

Solution. To expect $k = 10$ of the $n = 1000$ points inside a hypercube of edge length ℓ (with uniform density), the expected count is $n \cdot \ell^d$ (the hypercube’s volume ℓ^d times the point density, since the total volume is 1 and points are uniform). Set this equal to k :

$$n \ell^d = k \implies \ell^d = \frac{k}{n} = \frac{10}{1000} = 0.01 \implies \ell = (0.01)^{1/d}.$$

Computing for each d :

- $d = 1$: $\ell = 0.01^1 = 0.01$ (i.e. 1% of the axis range — a tiny, intuitive local neighborhood).
- $d = 2$: $\ell = 0.01^{1/2} = 0.1$ (10% of each axis — already a noticeably larger fraction).
- $d = 5$: $\ell = 0.01^{1/5} = 10^{-2/5} \approx 0.398$ (about 40% of each axis!).
- $d = 10$: $\ell = 0.01^{1/10} = 10^{-1/5} \approx 0.631$ (over 63% of the range along every axis).
- $d = 100$: $\ell = 0.01^{1/100} = 10^{-2/100} = 10^{-0.02} \approx 0.955$ (95.5% of the *entire* range along every single axis).

Interpretation. This is the curse of dimensionality made concrete: to capture just 10 neighbours out of 1000 points, the required “local” neighbourhood must span nearly the *entire* range of the feature space in high dimensions ($d = 100$: 95.5% of each axis!) — there is essentially no such thing as a small local neighbourhood in high-dimensional space with a fixed, realistic sample size. This is precisely why distance-based methods like k -NN degrade sharply as dimensionality grows (unless n grows exponentially with d to compensate, which is rarely feasible) and motivates dimensionality reduction (Chapter 3, Chapter 8) as a near-mandatory preprocessing step for such methods in high-dimensional settings.